

Trabajo: Técnicas de Reconocimiento de Patrones

Jorge Bengoa Pinedo

Bogurad Barański Barańska

Sergio Izquierdo Morona

7 de febrero de 2026

Índice

1. Introducción a la clasificación de imágenes térmicas	2
1.1. Redes Convolucionales...	2
1.2. Metodología Yolo	2
2. Fundamentos de funcionamiento de la red Yolo	3
2.1. Neurona de la red Yolo	3
2.2. Topología de la red	5
2.3. Reglas de entrenamiento	10
3. Desarrollo	10
4. Resultados	10

1. Introducción a la clasificación de imágenes térmicas

Usamos la version YOLO v5 [1] Explicar el estado del arte un poco de que hay y tal.... redes tal, este otro tipo de clasificación

1.1. Redes Convolucionales...

1.2. Metodología Yolo

2. Fundamentos de funcionamiento de la red Yolo

A continuación se desarrolla el funcionamiento de la red YOLO mediante la teoría de Marr para lo cuál es necesario describir el nivel implementacional y algorítmico. En este caso las redes neuronales artificiales se componen de 3 componentes que se describirán a continuación:

- Caracterización de la neurona.
- Definición de una topología de interconexión.
- Definición de unas reglas de entrenamiento.

2.1. Neurona de la red Yolo

La unidad fundamental de procesamiento en la arquitectura YOLO es la capa convolucional (típica de las CNN). En esta sección se describe el funcionamiento de dicha operación, tomando como caso de estudio la capa de entrada, analizando tanto el entrenamiento específico realizado para imágenes térmicas como la implementación por defecto, tal como se ilustra en la Figura 1.

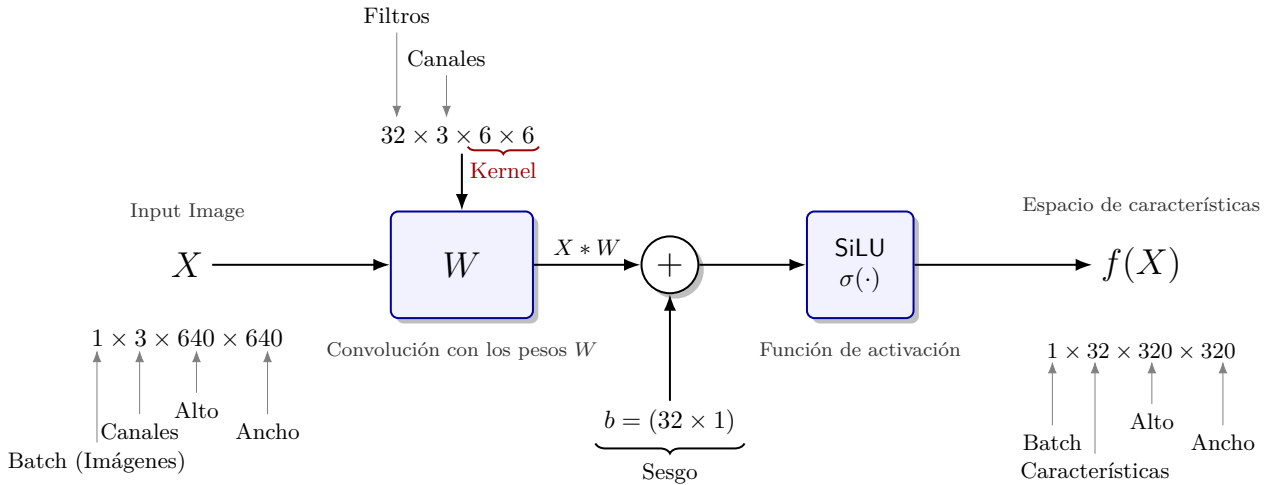


Figura 1: Estructura de la neurona a la entrada de la red YOLO.

La operación de esta neurona convolucional está definida por los siguientes hiperparámetros:

- **Número de filtros (Características C_{out}):** Determinado por la primera dimensión del tensor de pesos. Define cuántos mapas de características se generarán.
- **Canales de entrada (C_{in}):** Determinado por la segunda dimensión del tensor de pesos (e.g., 3 para RGB, 1 para imágenes térmicas).
- **Tamaño del Kernel (k):** Define las dimensiones espaciales de la ventana de convolución (últimas dos dimensiones de los pesos). En la arquitectura YOLO analizada, se emplean principalmente los tamaños (6,6) para la entrada, y (3,3) o (1,1) para capas profundas.
- **Padding (P):** Parámetro de relleno utilizado para controlar las dimensiones espaciales de la salida e indicar los bordes al llenarlos de ceros. En YOLO se utilizan típicamente valores de (2,2), (1,1), o (0,0).
- **Stride (S):** Controla el paso o desplazamiento del filtro sobre la imagen. Un $S = 1$ mueve el filtro píxel a píxel (preservando resolución), mientras que $S = 2$ lo desplaza cada dos píxeles, reduciendo la dimensión espacial a la mitad.
- **Dilatación (d):** Controla la separación entre los puntos que analiza el kernel (también conocida como *atrous convolution* cuando $d > 1$). En esta implementación de YOLO, el parámetro es (1,1), lo que indica una convolución estándar compacta sin huecos entre los elementos del kernel.

- **Agrupación (g):** Controla la conectividad entre los canales de entrada y salida. Dado que en YOLO se utiliza $g = 1$, se realiza una convolución estándar donde todos los canales de entrada contribuyen al cálculo de cada filtro de salida.

A partir de estos hiperparámetros, es posible determinar las dimensiones espaciales del mapa de características de salida. Dado que se contempla el factor de dilatación, se debe utilizar la relación generalizada descrita por [2].

$$\text{Salida} = \left\lfloor \frac{\text{Entrada} + 2 \cdot P - [k + (k - 1)(d - 1)]}{S} \right\rfloor + 1 \quad (1)$$

En cuanto a la activación de cada neurona, el proceso consiste en aplicar la función SiLU (*Sigmoid Linear Unit*) al resultado de la operación lineal (convolución más sesgo). Matemáticamente, la activación $f(X)$ se expresa como:

$$f(X) = \underbrace{[(W * X) + b]}_z \cdot \underbrace{\sigma([(W * X) + b])}_z \quad (2)$$

donde $\sigma(z) = \frac{1}{1+e^{-z}}$ es la función sigmoide logística. Se utiliza esta función de activación porque permite el cálculo de gradientes y solo permite valores negativos pequeños para mejorar el flujo de gradientes [3, 4], se ilustra en la Figura 2.

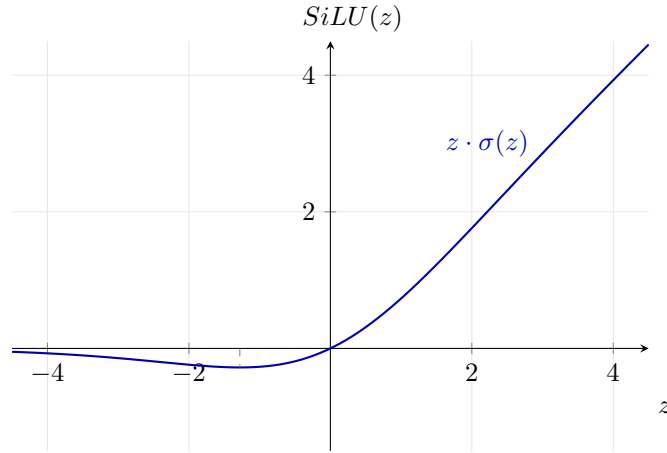


Figura 2: Representación gráfica de la función de activación SiLU.

Por otro lado, la operación de convolución discreta 2D ($W * X$) en el contexto de aprendizaje profundo se implementa técnicamente como una correlación cruzada (sin voltear el kernel) pues solo se busca conocer los pesos y no aplicar un filtro con una topología concreta. Para un filtro de salida específico (c_{out}) en la posición espacial (h, w), la operación se define formalmente como [2]:

$$(W * X)(c_{out}, h, w) = \sum_{l=0}^{\frac{c_{in}}{g}-1} \sum_{i=0}^{K_H-1} \sum_{j=0}^{K_W-1} W(c_{out}, l, i, j) \cdot X(l, h \cdot S + i \cdot d, w \cdot S + j \cdot d) \quad (3)$$

2.2. Topología de la red

En esta sección tras analizar los artículos principales de los creadores de Yolo [5][6][7][8] y el diagrama de la red mediante la herramienta *Netron* donde se introduce la red en formato *.onnx* y se analiza su diagrama que tiene una forma similar al de la figura 3.

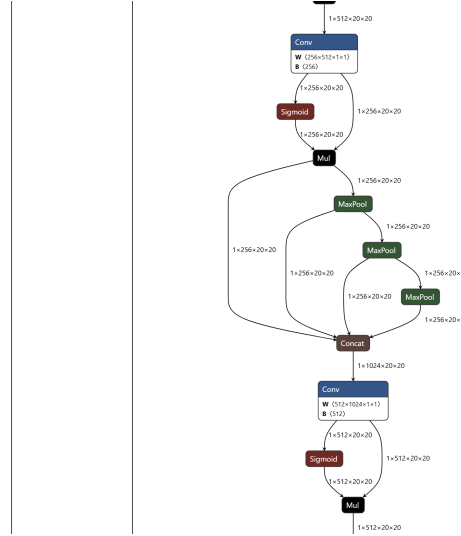


Figura 3: Ejemplo de parte de la red Yolo en la aplicación *Netron*.

Uno de los bloques que más se repite a lo largo de la arquitectura de la red Yolo es el bloque ResNet, cuya estructura se encuentra representada en la figura 4. La filosofía detrás del diseño de este bloque viene detallada en el artículo [9], y su objetivo principal se mejoró el rendimiento de las redes neuronales profundas cuando se acumulan múltiples capas.

La problemática surge durante el entrenamiento: al acumular múltiples capas consecutivas, los gradientes tienden a explotar. Antes de la existencia de este bloque, esto se mitigaba mediante la normalización en capas intermedias, pero aun así se observaba una degradación en la convergencia. Para entender esta degradación, se puede entender la operación de convolución como una aplicación identidad con cierto ruido superpuesto que representa las características a extraer. Por tanto, mediante una conexión lineal directa se optimiza el comportamiento del método de optimización utilizado, mejorando así la convergencia.

La premisa de que los bloques se comportan como aplicaciones lineales con ruido se demuestra empíricamente en el propio artículo [9]. Aunque se trata más de una heurística que de un argumento formal, en casos reales, como el de la figura 5, presenta un mejor comportamiento que simplemente acumular capas, y solo sería problemático si el objetivo del bloque fuera aproximar la función nula.

La interpretación del bloque ResNet como una aplicación lineal con ruido se deduce fácilmente a partir de las ecuaciones del modelo:

$$y = \mathcal{H}(x) = \mathcal{F}(x) + x \quad (4)$$

donde y es la salida de aplicar $\mathcal{H}$ a la entrada x que se descompone entre una función que representa la convolución $\mathcal{F}(x)$ y la aplicación lineal directa. Por tanto, se puede asemejar el funcionamiento de las capas convolucionales como un extractor del ruido o características.

Profundizando en el detalle del propio bloque, este está formado por dos capas, donde la primera se encarga de procesar los canales para que la segunda simplemente extraiga las características de los anteriores.

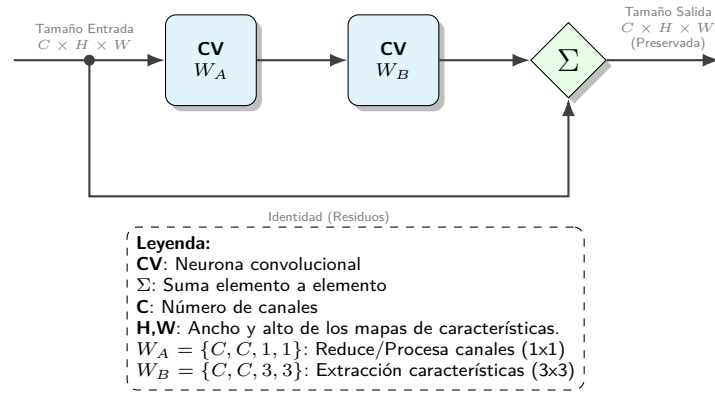


Figura 4: Arquitectura de un bloque ResNet.

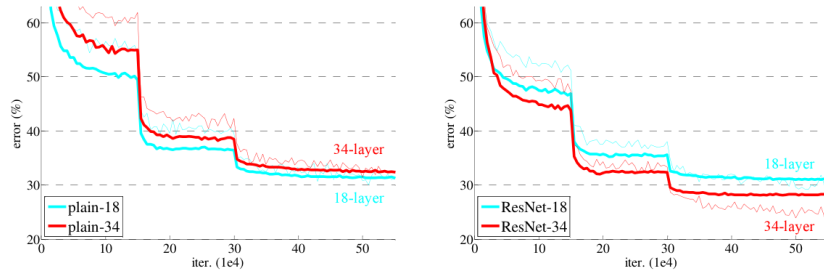


Figura 5: Representación del error de entrenamiento en función del número de capas. A la izquierda se representa una arquitectura con capas apiladas y a la derecha una arquitectura de capas apiladas utilizando el bloque ResNet.

Para elaborar el bloque 6 se usa el artículo [10]

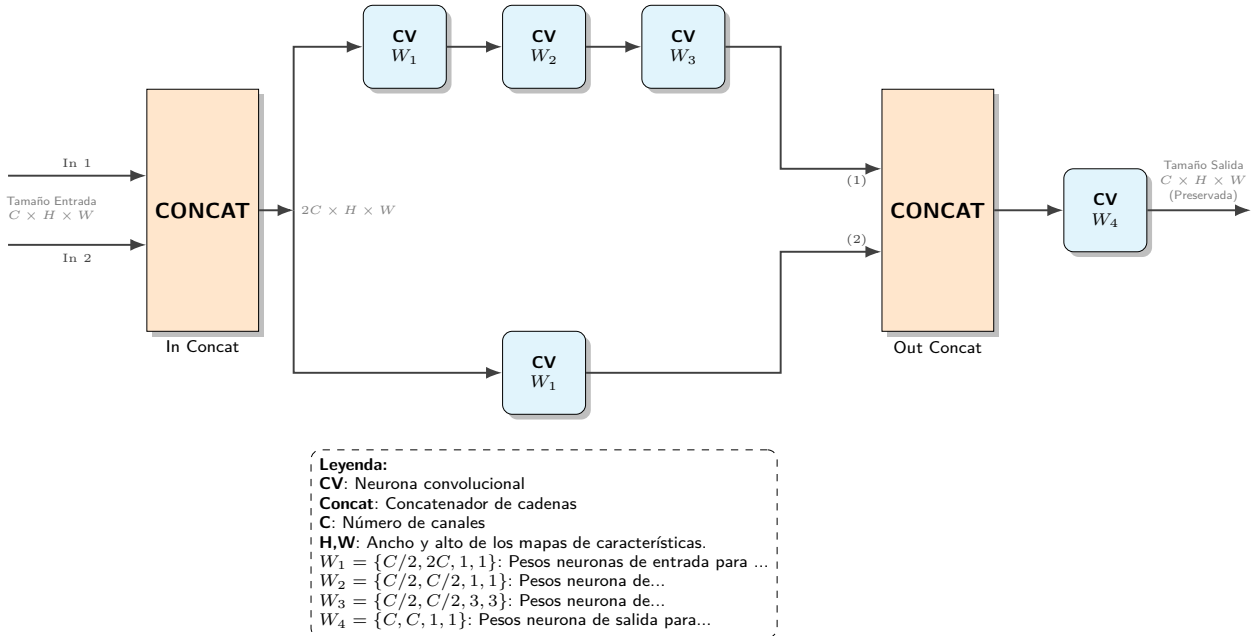


Figura 6: Architecture of the CSP (YOLOv5 C3 Module)

Para elaborar el bloque 7 se usa el artículo [10]

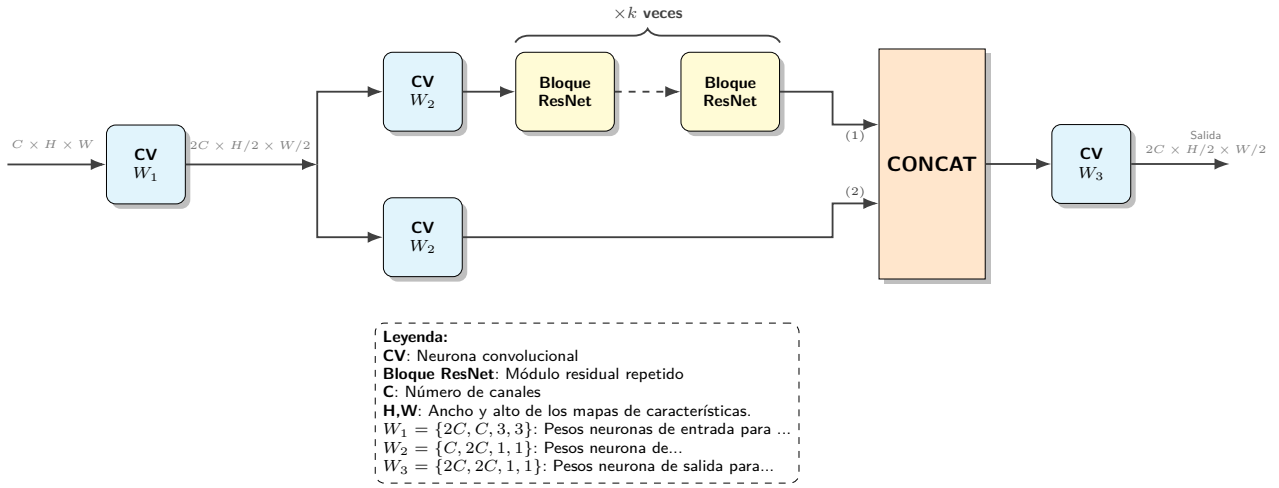


Figura 7: Arquitectura del Módulo CSPResNe(X)t o C3-k

To enlarge the receptive field and fuse features at different scales, we employ the Spatial Pyramid Pooling - Fast (SPPF) module. This is an optimized implementation of the original Spatial Pyramid Pooling (SPP) proposed by He et al. [11]. While the original SPP processes features through parallel pooling windows of sizes $k = \{1, 5, 9, 13\}$, SPPF achieves the same mathematical result by sequentially passing the input through multiple 5×5 max-pooling layers. This sequential structure, introduced in the YOLOv5 architecture [1], significantly reduces computational latency while preserving the multi-scale feature fusion capability.”⁸

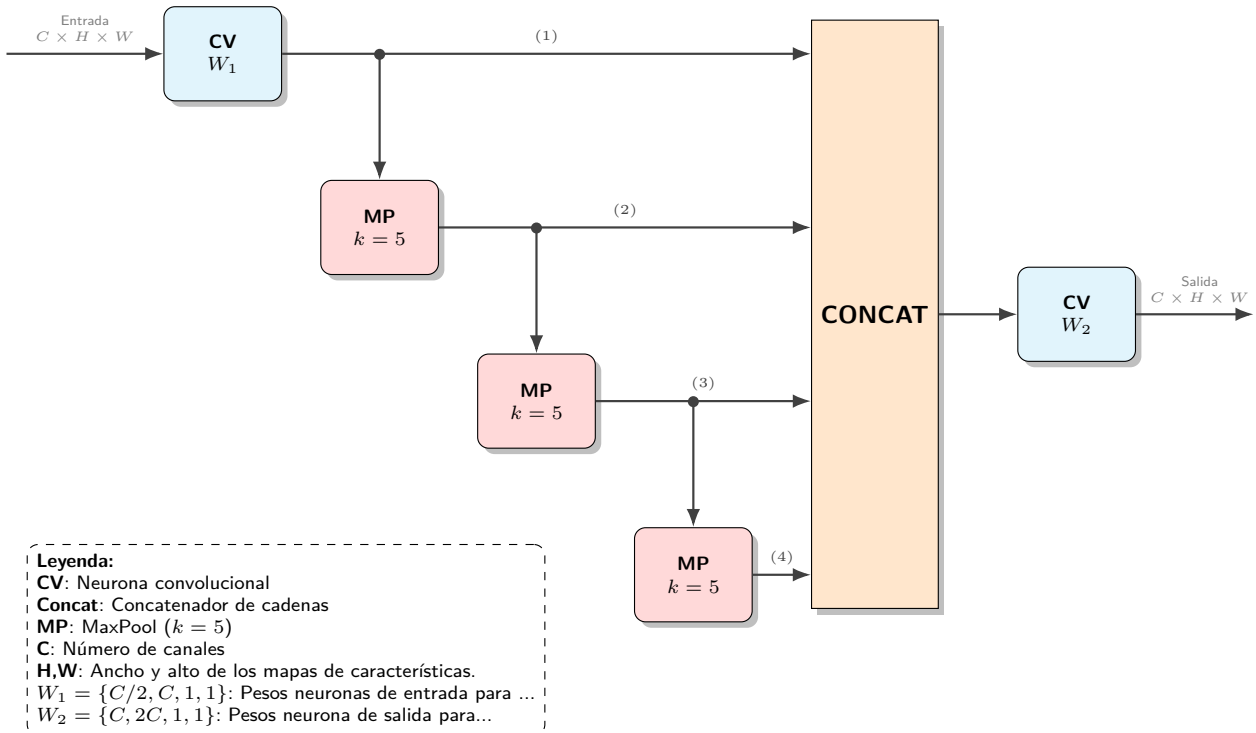


Figura 8: Arquitectura SPP en Cascada

La estructura de la cabeza 9 puede explicarse directamente a través de los artículos de Yolo

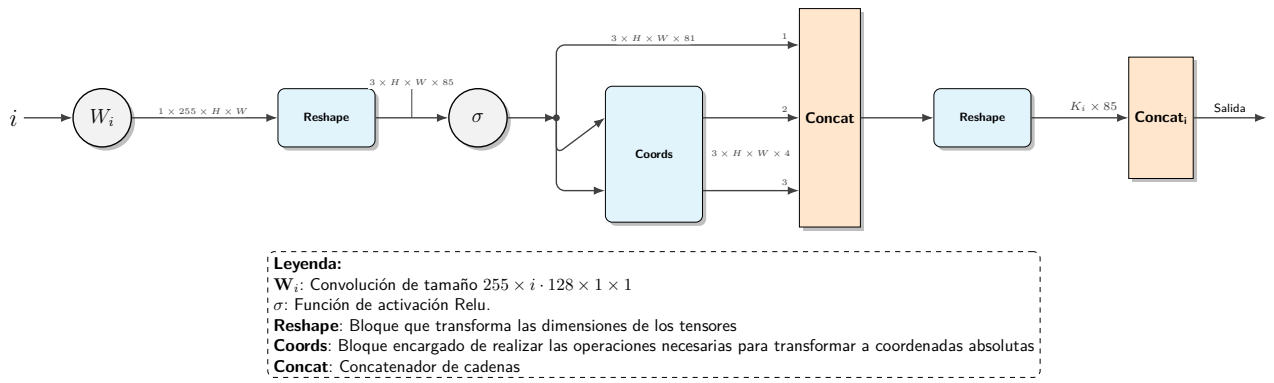


Figura 9: Detalle de la Cabeza de Detección (YOLO Head)

La implementación final se obtiene del análisis de la estructura 10

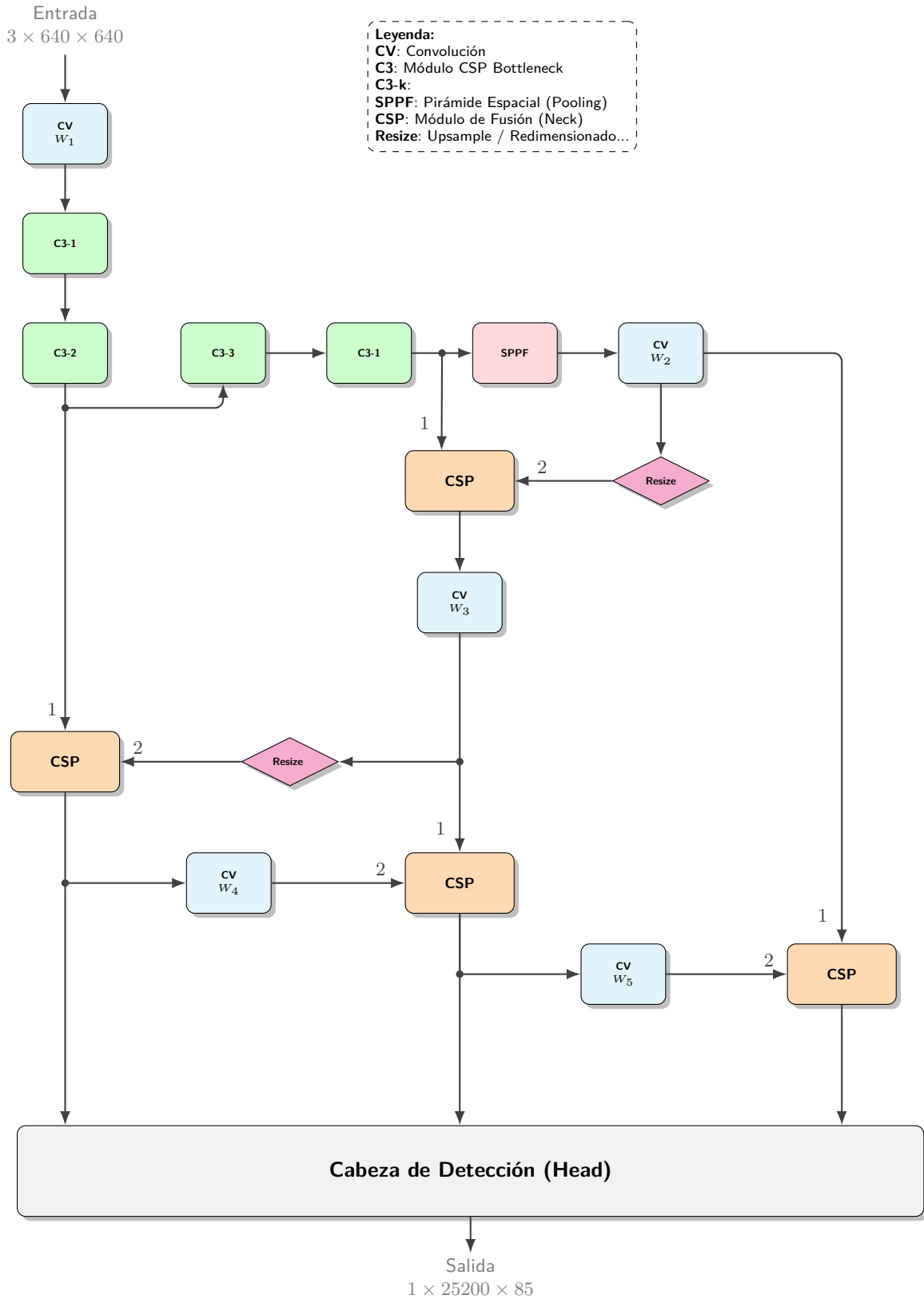


Figura 10: Arquitectura Global YOLO: Inicio Vertical y Flujo Descendente

2.3. Reglas de entrenamiento

3. Desarrollo

4. Resultados

Referencias

- [1] G. Jocher, “YOLOv5 by Ultralytics,” 5 2020.
- [2] V. Dumoulin and F. Visin, “A guide to convolution arithmetic for deep learning,” 2018.
- [3] S. Elfving, E. Uchibe, and K. Doya, “Sigmoid-weighted linear units for neural network function approximation in reinforcement learning,” 2017.
- [4] P. Ramachandran, B. Zoph, and Q. V. Le, “Searching for activation functions,” 2017.
- [5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” 2016.
- [6] J. Redmon and A. Farhadi, “Yolo9000: Better, faster, stronger,” 2016.
- [7] J. Redmon and A. Farhadi, “Yolov3: An incremental improvement,” 2018.
- [8] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “Yolov4: Optimal speed and accuracy of object detection,” 2020.
- [9] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” 2015.
- [10] C.-Y. Wang, H.-Y. M. Liao, I.-H. Yeh, Y.-H. Wu, P.-Y. Chen, and J.-W. Hsieh, “Cspnet: A new backbone that can enhance learning capability of cnn,” 2019.
- [11] K. He, X. Zhang, S. Ren, and J. Sun, *Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition*, p. 346–361. Springer International Publishing, 2014.